INSTITUTO POLITÉCNICO NACIONAL
UPIICSA
INGENIERÍA DE PRUEBAS

Desarrollo de la App

Animal Crossing

PROF.: Ramon Cruz Martínez

ALUMNOS:

- **Martínez Lozano Armando**
- **Medina Gómez Jonathan Ramón**
- **Mendoza Olvera Diego Alejandro**
- **Paz Abedoy Christian**

GRUPO: 6NV61

Definición del Proyecto

El software consiste en una aplicación móvil para la contratación de personas que puedan pasear a mascotas (perros), permitiendo al dueño de la mascota comunicarse y monitorear al paseador.

Objetivos principales

- Permitir a los usuarios crear perfiles en dónde puedan seleccionar uno de dos roles: dueño y paseador.
- Permitir a los dueños poder registrar, visualizar y monitorear a las mascotas registradas, así como poder escoger entre las rutas y paseadores registrados para que paseen sus mascotas.
- Los paseadores podrán escoger entre rechazar o aceptar las solicitudes de paseo de los dueños.
- Los dueños y paseadores tendrán la capacidad de comunicarse entre sí.

Especificaciones y Requerimientos

Para el desarrollo de la aplicación:

- Android Studio Otter | 2025.2.1 Patch 1
    - Lenguaje de Programación: Kotlin
- Google Cloud
    - Maps SDK for Android
    - Routes API
- SO: Windows 11
    - 24H2

Para el entorno de simulación:

- *Medium Phone*
    - API: 36.0 “Baklava”
    - SO: Android 16.0
    - Servicios: Google Play Store
    - Imagen del sistema: Google Play Intel x86_64 Atom System Image

Planificación y Alcance

Como se mencionó, la aplicación deberá permitir crear perfiles en donde los usuarios puedan seleccionar un rol y con base a este desempeñar ciertas actividades; en el caso de los dueños, estos podrán registrar mascotas y seleccionar dichas mascotas para que un paseador (igualmente seleccionado por el dueño) los lleve de paseo por cierta ruta

(también seleccionada por el dueño), al finalizar el paseo el dueño podrá calificar al paseador.

Por otro lado, el paseador recibe las solicitudes de paseo que el dueño envío, las podrá aceptar o rechazar. De haberlas aceptado, el paseador tiene la opción de iniciar el paseo y finalizarlo, debiendo entregar evidencia fotográfica de que la mascota fue devuelta correctamente al dueño.

Este es el flujo de trabajo básico mediante el cual se basa la aplicación.

Algunos aspectos que caen fuera de lo que se prevé que la aplicación realice son la capacidad de crear rutas y tarifas personalizadas; las rutas disponibles en la aplicación no pueden ser modificadas (a menos de que el dueño decida lo contrario, en ese caso, la responsabilidad cae sobre el dueño y el paseador en caso de aceptar), así como la tarifa de cada una de las rutas (igualmente, si se decide cambiarlo, recae enteramente en el dueño y paseador), es decir, no existe ningún medio por el cual se puede realizar modificación alguna o crear nuevas rutas; todos los cambios serán hechos por los *administradores* de la aplicación.

Por otro lado, servicios que pueden verse como ligados o cercanos al propósito principal de la aplicación, como servicios veterinarios, de guardería, adopción o relacionados, tampoco serán considerados. El enfoque es el de facilitar al dueño de una mascota encontrar personas que puedan pasearlas.

Arquitectura del sistema

Jerarquía

La manera en cómo se organiza el sistema es mediante módulos, cada uno realizando una función específica, los cuales se organizan de la siguiente manera:

- Activities
    - Fragments
        - Adapters
        - Classes
        - Database
            - Entity
                - Dao
    - Layouts
        - Drawable
        - Values

El nivel principal es Activities, que son los procesos principales de la aplicación, es decir procesos los cuales son fundamentales que se ejecuten para que la aplicación pueda funcionar correctamente.

El siguiente nivel está conformado por los Fragments y Layouts. Los Fragments proporcionan funcionalidad al sistema y realizan funciones relevantes pero que no se consideran críticos para que la aplicación funcione con normalidad. Este se apoya del siguiente nivel, conformado por los Adapters, los cuales adaptan la información de la base de datos para su correcto uso en Fragments y su correcto despliegue en Layouts; Classes crea clases que facilitan la relación de objetos dentro de la aplicación con los Fragments; y Database es la base de datos, Entity siendo las tablas que conforman la base de datos y Dao siendo las sentencias (Querys) que permiten hacer transacciones con la base de datos.

Layouts son las interfaces de usuario las cuales se apoya de Drawables, que son varios recursos gráficos que se despliegan a lo largo del sistema, y Values, que son recursos varios, como los colores y texto que se muestran en la interfaz.

Módulos

Los módulos a detalle que conforman cada uno de los niveles de la jerarquía son:

Activities	
Nombre	Función
MainActivity	Pantalla de Login y paso de datos de *UserName, UserMail* y *UserRole* para el funcionamiento de los Fragments.
MainMenu	Carga el menú principal y funge como la pantalla principal a reemplazar por los Fragments.
Register	Pantalla de Registro para que el usuario pueda introducir sus datos y darse de alta dentro de la aplicación.

Fragments	
Nombre	Función
Account	Carga los datos del usuario den sesión para que los pueda visualizar y/o cambiar. También permite cerrar sesión.
ChooseRoute	Carga las rutas disponibles para que el dueño pueda seleccionar alguna.

Home	Para el dueño, carga las mascotas que tenga registradas para que pueda seleccionarlos y se pueda iniciar el flujo para solicitar un paseo. Para el paseador, carga las solicitudes que haya aceptado y seleccionarlas para que se pueda inicializar el paseo.
Message	Sección de mensajería interna.
Pet	Para el dueño, carga una pantalla para que pueda registrar una mascota. Para el paseador, carga las solicitudes de paseo de los dueños para que pueda aceptarlas o rechazarlas.
Rating	Permite al dueño poner una calificación al paseador una vez se haya finalizado el paseo.
Route	Carga la lista de rutas disponibles.
RouteMapFragment	Carga la vista previa del mapa con base a la ruta seleccionada.
WalkerList	Carga la lista de paseadores disponibles.
WalkerProfile	Carga la información relevante del paseador para que el dueño la pueda visualizar.
WalkInProgress	Carga la vista donde se pueda finalizar el paseo si se sube una foto de la mascota siendo entregado al dueño como evidencia.

Adapters	
Nombre	Función
AcceptedRequestAdapter	Da formato a las solicitudes de paseo aceptadas.
PetAdapter	Da formato a la información de una mascota.
RouteAdapter	Da formato a la información de una ruta.
WalkerAdapter	Da formato a la información de los paseadores.
WalkerRequestAdapter	Da formato a las solicitudes de paseo enviadas por el dueño.

Classes	
Nombre	Función
UserWithWalkerData	Permite seleccionar información específica de la tabla de *userEntity*.
WalkerWithRating	Permite seleccionar información específica de la tabla de *walkerEntity* y *userEntity*.

Databases	
Nombre	Función
AppDatabase	Crea la base de datos con las *Entity*'s y *Dao*'s creados.
dataBaseProvider	Inicializa la base de datos para su uso en la aplicación.
PredefinedRoutes	Información de las rutas predeterminadas.

Entity	
Nombre	Función
conversationEntity	Tabla de las conversaciones *(ownerEmial, walkerEmail, date)*
messageEntity	Tabla de los mensajes de las conversaciones *(userEmisor, walkerReciever, message, date, isRead)*
petEntity	Tabla de las mascotas *(name, breed, description, photoUri, age, ownerEmail)*
routeEntity	Tabla de las rutas *(latitud, longitude, distance, duration)*
userEntity	Tabla de los usuarios *(userEmail, password, name, address, telephoneNumber, role)*
walkerEntity	Tabla de los paseadores *(walkerEmail, ratingSum, ratingCount, ratingAverage)*
walkerRequestEntity	Tabla de las solicitudes de paseo *(id, walkerEmail, ownerEmail, petId, petName, routeName, status)*

Dao	
Nombre	Función
petDao	Transacciones de la tabla petEntity
userDao	Transacciones de la tabla userEntity
walkerDao	Transacciones de la tabla walkerEntity
walkerRequestDao	Transacciones de la tabla walkerRequestEntity

Layout	
Nombre	Función
fragment_account	Visualización de la pantalla de cuenta del usuario en sesión.
fragment_choose_route	Muestra la lista de rutas disponibles.
fragment_home	Para el dueño, muestra la lista de mascotas registradas.

	Para el paseador, muestra la lista de solicitudes de paso aceptadas.
fragment_message	Muestra los mensajes entre dueño y paseador.
fragment_pet	Para el dueño, se muestra la pantalla de registro de mascota. Para el paseador, se muestran las solicitudes de paseo.
fragment_rating	Muestra la pantalla para calificar al paseador.
fragment_route	Muestra las rutas disponibles.
fragment_route_map	Muestra la previsualización del mapa seleccionado.
fragment_walk_in_progress	Muestra la pantalla del paseo iniciado.
fragment_walker_list	Muestra la lista de paseadores disponibles para ser seleccionados por el dueño.
fragment_walker_profile	Muestra el resumen de los datos para enviar la solicitud de paseo.
item_accepted_requests	Tarjeta con los datos básicos de las solicitudes aceptadas.
item_pet_card	Tarjeta con los datos básicos de la mascota.
item_route_card	Tarjeta con los datos básicos de la ruta.
item_walk_request	Tarjeta con los datos básicos de la solicitud de paseo.
item_walker_card	Tarjeta con los datos básicos del paseador.
login	Muestra la pantalla para iniciar sesión.
main_menu	Muestra la barra de exploración y carga un layout vacío que será reemplazado por los Fragments.
register	Muestra la pantalla para crear un perfil en la aplicación.

Drawable	
Nombre	Función
account	Imágen (xml)
bottom_background	Imágen (xml)
dog_placeholder	Imágen (jpg)
edit	Imágen (xml)
home	Imágen (xml)
ic_launcher_background	Imágen (xml)
ic_launcher_foreground	Imágen (xml)
logo	Imágen (png)
message	Imágen (xml)
pet	Imágen (xml)
route	Imágen (xml)

Values	
Nombre	Función
colors	Colores que conforman el tema de la aplicación.
string	Texto que se despliega en los layouts de la aplicación.
themes	Tema de la aplicación.

Codificación de los Módulos

Por cuestiones de practicidad, no se pondrá el código completo de cada uno de los módulos, sino que se colocará el código relevante de cada módulo, es decir, se evitará colocar el *package* (ya que este siempre será package *com.example.animalcrossing*) y se evitará colocar los *imports* de cada módulo (ya que estos son muy similares entre sí), además de que Android Studio tiene el beneficio de autogenerar los *imports* al momento de introducir el código en el IDE.

El orden en el que mostrará la codificación de cada módulo es con base a la jerarquía establecida.

La codificación de cada módulo es la siguiente:

MainActivity.kt

```
class MainActivity : ComponentActivity() {


    private lateinit var loginButton: Button
    private lateinit var registerButton: Button


    private lateinit var userEmailInput: EditText
    private lateinit var userPasswordInput: EditText


    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.login)


        val session = SessionManager(this)
        val savedEmail = session.getUser()


        if(savedEmail != null) {
            val intent = Intent(this, MainMenu::class.java)
            startActivity(intent)
            finish()
        }


        userEmailInput = findViewById(R.id.userEmail)
        userPasswordInput = findViewById(R.id.userPassword)


        loginButton = findViewById(R.id.buttonLogin)
        registerButton = findViewById(R.id.buttonRegister)


        loginButton.setOnClickListener {
            val inputEmail = userEmailInput.text.toString()
            val inputPassword = userPasswordInput.text.toString()


            login(inputEmail, inputPassword)
        }
```

```
        registerButton.setOnClickListener {
            val intentRegister = Intent(this@MainActivity, Register::class.java)
            startActivity(intentRegister)
        }
    }

    private fun login(email: String, password: String) {
        val db = dataBaseProvider.getDatabase(this)

        CoroutineScope(Dispatchers.IO).launch {
            val user = db.userDao().getUserById(email)

            withContext(Dispatchers.Main) {
                if(user == null || user.password != password) {
                    Toast.makeText(this@MainActivity, "Usuario o contraseña incorrectos", Toast.LENGTH_SHORT).show()
                } else {
                    val intent = Intent(this@MainActivity, MainMenu::class.java).apply {
                        putExtra("UserName", user.name)
                        putExtra("UserEmail", user.userEmail)
                        putExtra("UserRole", user.role)
                    }
                    startActivity(intent)
                }
            }
        }
    }
}
```

MainMenu.kt

```
class MainMenu : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.main_menu)

        val userName = intent.getStringExtra("UserName") ?: "Usuario"
        val userEmail = intent.getStringExtra("UserEmail") ?: "Correo"
        val userRole = intent.getStringExtra("UserRole") ?: "Role"
        val bottomNav: BottomNavigationView = findViewById(R.id.bottomNavigationView)

        if(savedInstanceState == null) {
            bottomNav.selectedItemId = R.id.bottom_home
            loadFragment(Home.newInstance(userName, userEmail, userRole))
        }

        bottomNav.setOnItemSelectedListener { item ->
            val fragment = when(item.itemId) {
                R.id.bottom_account -> Account()
                R.id.bottom_pet -> Pet.newInstance(userEmail, userRole)
                R.id.bottom_home -> Home.newInstance(userName, userEmail, userRole)
                R.id.bottom_route -> Route()
                R.id.bottom_message -> Message()
                else -> null
            }

            fragment?.let {
                loadFragment(it)
                true
            } ?: false
        }
    }

    private fun loadFragment(fragment: Fragment) {
        supportFragmentManager.beginTransaction()
            .replace(R.id.fragment_contanier, fragment)
            .commit()
    }
}
```

Register.kt

```
class Register : ComponentActivity() {
    private lateinit var registerButton: Button

    private lateinit var radioGroup: RadioGroup

    private lateinit var userEmailInput: EditText
    private lateinit var userPasswordInput: EditText
    private lateinit var userNameInput: EditText
    private lateinit var userPhoneInput: EditText
    private lateinit var userAddressInput: EditText
```

```
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.register)


        userEmailInput = findViewById(R.id.userEmailRegister)
        userPasswordInput = findViewById(R.id.userPasswordRegister)
        userNameInput = findViewById(R.id.userNameRegister)
        userPhoneInput = findViewById(R.id.userPhoneRegister)
        userAddressInput = findViewById(R.id.userAddressRegister)


        radioGroup = findViewById(R.id.radioGroupRole)


        registerButton = findViewById(R.id.button)


        registerButton.setOnClickListener {
            val inputEmail = userEmailInput.text.toString()
            val inputPassword = userPasswordInput.text.toString()
            val inputName = userNameInput.text.toString()
            val inputAddress = userAddressInput.text.toString()
            val inputPhone = userPhoneInput.text.toString()


            val selectedRole = radioGroup.checkedRadioButtonId
            val role = when(selectedRole) {
                R.id.radioButtonOwner -> "Owner"
                R.id.radioButtonWalker -> "Walker"
                else -> ""
            }


            if(inputEmail.isBlank() || inputPassword.isBlank() || inputName.isBlank() || inputAddress.isBlank() || inputPhone.isBlank() || role.isBlank()){
                Toast.makeText(this, "Por favor, llene todos los campos", Toast.LENGTH_SHORT).show()
                return@setOnClickListener
            }


            register(inputEmail, inputPassword, inputName, inputAddress, inputPhone, role)
        }
    }


    private fun register(email: String, password: String, name: String, address: String, phone: String, role: String) {
        val db = dataBaseProvider.getDatabase(this)
        val user = userEntity(email, password, name, address, phone, role)
        CoroutineScope(Dispatchers.IO).launch {
            db.userDao().insertUser(user)
        }


        Toast.makeText(this, "Usuario registrado correctamente", Toast.LENGTH_SHORT).show()


        val intent = Intent(this@Register, MainActivity::class.java)
        startActivity(intent)
    }
}
```

Account.kt

```
class Account : Fragment() {
    private lateinit var editEmail: EditText
    private lateinit var editPassword: EditText
    private lateinit var editName: EditText
    private lateinit var editAddress: EditText
    private lateinit var editTelephone: EditText
    private lateinit var saveButton: Button
    private lateinit var logoutButton: Button
    private lateinit var userEmail: String


    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)


        userEmail = requireActivity()
            .intent
            .getStringExtra("UserEmail")
            ?: ""
    }


    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        val view = inflater.inflate(R.layout.fragment_account, container, false)
```

```
        editEmail = view.findViewById(R.id.userEmailReadOnly)
        editPassword = view.findViewById(R.id.editTextUserPassword)
        editName = view.findViewById(R.id.editTextUserName)
        editAddress = view.findViewById(R.id.editTextUserAddress)
        editTelephone = view.findViewById(R.id.editTextUserTelephone)


        saveButton = view.findViewById(R.id.buttonSaveChanges)
        logoutButton = view.findViewById(R.id.buttonLogout)


        editEmail.setText(userEmail)
        editEmail.isEnabled = false


        getUserData()


        saveButton.setOnClickListener {
            updateUserData()
        }


        logoutButton.setOnClickListener {
            logout()
        }


        return view
    }


    private fun getUserData() {
        val db = dataBaseProvider.getDatabase(requireContext())


        CoroutineScope(Dispatchers.IO).launch {
            val user = db.userDao().getUserById(userEmail)


            if (user != null) {
                launch(Dispatchers.Main) {
                    editName.setText(user.name)
                    editPassword.setText(user.password)
                    editAddress.setText(user.address)
                    editTelephone.setText(user.telephoneNumber)
                }
            }
        }
    }


    private fun updateUserData() {
        val newPassword = editPassword.text.toString()
        val newName = editName.text.toString()
        val newAddress = editAddress.text.toString()
        val newTelephoneNumber = editTelephone.text.toString()


        val db = dataBaseProvider.getDatabase(requireContext())


        if(newPassword.isBlank() || newName.isBlank() || newAddress.isBlank() || newTelephoneNumber.isBlank()) {
            Toast.makeText(requireContext(), "Complete todos los campos", Toast.LENGTH_SHORT).show()
            return
        }
        CoroutineScope(Dispatchers.IO).launch {
            val rows = db.userDao().updateUserById(userEmail, newPassword, newName, newAddress, newTelephoneNumber)


            CoroutineScope(Dispatchers.Main).launch {
                if(rows > 0) {
                    Toast.makeText(requireContext(), "Datos actualizados", Toast.LENGTH_SHORT).show()
                } else {
                    Toast.makeText(requireContext(), "Ocurrió un error", Toast.LENGTH_SHORT).show()
                }
            }
        }
    }
    private fun logout() {
        val session = SessionManager(requireContext())
        session.clearSession()


        val intent = Intent(requireContext(), MainActivity::class.java)
        intent.flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
        startActivity(intent)
    }
}
```

ChooseRoute.kt

```
class ChooseRoute : Fragment() {
    companion object {
        private const val ARG_PET_ID = "petId"
        private const val ARG_PET_NAME = "petName"
        private const val ARG_USER_EMAIL = "userEmail"


        fun newInstance(petId: Int, petName: String, userEmail: String): ChooseRoute {
            val fragment = ChooseRoute()
            val args = Bundle()
            args.putInt(ARG_PET_ID, petId)
            args.putString(ARG_PET_NAME, petName)
            args.putString(ARG_USER_EMAIL, userEmail)
            fragment.arguments = args
            return fragment
        }
    }


    private lateinit var routeRecycler: RecyclerView
    private lateinit var adapter: RouteAdapter


    private var petId: Int = -1
    private var petName: String = ""
    private var userEmail: String = ""


    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
    }


    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        val view = inflater.inflate(R.layout.fragment_choose_route, container, false)


        petId = arguments?.getInt(ARG_PET_ID) ?: -1
        petName = arguments?.getString(ARG_PET_NAME) ?: ""
        userEmail = arguments?.getString(ARG_USER_EMAIL) ?: ""


        routeRecycler = view.findViewById(R.id.routeRecycler)
        routeRecycler.layoutManager = LinearLayoutManager(requireContext())


        adapter = RouteAdapter(PredefinedRoutes.allRoutes) { selectedRoute ->
            navigateToWalkerList(selectedRoute)
        }


        routeRecycler.adapter = adapter


        return view
    }


    private fun navigateToWalkerList(route: PredefinedRoute) {


        val db = dataBaseProvider.getDatabase(requireContext())


        CoroutineScope(Dispatchers.IO).launch {
            val user = db.userDao().getUserById(userEmail)


            launch(Dispatchers.Main) {
                if (user == null) {
                    Toast.makeText(requireContext(), "Usuario no encontrado", Toast.LENGTH_SHORT).show()
                    return@launch
                }


                val fragment = WalkerList.newInstance(
                    route = route,
                    petId = petId,
                    petName = petName,
                    user = user
                )


                parentFragmentManager.beginTransaction()
                    .replace(R.id.fragment_contanier, fragment)
                    .addToBackStack(null)
                    .commit()
```

```
            }
        }
    }
}
```

Home.kt

```
class Home : Fragment() {


    private lateinit var petRecyclerView: RecyclerView
    private lateinit var petAdapter: PetAdapter
    private lateinit var emptyMessage: TextView
    private lateinit var acceptedCard: CardView
    private lateinit var acceptedPetName: TextView
    private lateinit var acceptedRouteName: TextView
    private lateinit var btnStartWalk: Button
    private lateinit var userEmail: String
    private lateinit var userName: String
    private lateinit var userRole: String


    companion object {
        private const val ARG_USERNAME = "UserName"
        private const val ARG_USEREMAIL = "UserEmail"
        private const val ARG_USERROLE = "UserRole"


        fun newInstance(userName: String, userEmail: String, userRole: String): Home {
            val fragment = Home()
            val args = Bundle()
            args.apply {
                putString(ARG_USERNAME, userName)
                putString(ARG_USEREMAIL, userEmail)
                putString(ARG_USERROLE, userRole)
            }
            fragment.arguments = args
            return fragment
        }
    }


    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        val view = inflater.inflate(R.layout.fragment_home, container, false)


        userName = arguments?.getString(ARG_USERNAME) ?: "Usuario"
        userEmail = arguments?.getString(ARG_USEREMAIL) ?: ""
        userRole = arguments?.getString(ARG_USERROLE) ?: "Owner"


        val userGreeting: TextView = view.findViewById(R.id.userGreeting)
        userGreeting.text = "Bienvenido, $userName"


        emptyMessage = view.findViewById(R.id.emptyMessage)
        petRecyclerView = view.findViewById(R.id.petRecycleViewer)
        petRecyclerView.layoutManager = LinearLayoutManager(requireContext())
        petAdapter = PetAdapter(
            pets = emptyList(),
            onPetSelected = { pet -> openChooseRoute(pet, userEmail) }
        )
        petRecyclerView.adapter = petAdapter


        acceptedCard = view.findViewById(R.id.acceptedRequestCard)
        acceptedPetName = view.findViewById(R.id.acceptedRequestPetName)
        acceptedRouteName = view.findViewById(R.id.acceptedRequestRouteName)
        btnStartWalk = view.findViewById(R.id.startWalkButton)


        loadUserRole()


        return view
    }


    private fun loadUserRole() {
        if(userRole == "Owner") {
            showOwnerUI()
            loadPets()
            observeWalkInProgressForOwner()
            observeFinishedWalkForOwner()
        } else {
            showWalkerUI()
            loadAcceptedRequest()
        }
    }
```

```
private fun loadPets() {
    val db = dataBaseProvider.getDatabase(requireContext())

    CoroutineScope(Dispatchers.IO).launch {
        db.petDao().getPetsByOwner(userEmail).collect { petList ->
            withContext(Dispatchers.Main) {
                petAdapter.updateData(petList)
                emptyMessage.visibility =
                    if (petList.isEmpty()) View.VISIBLE else View.GONE
            }
        }
    }
}

private fun showOwnerUI() {
    acceptedCard.visibility = View.GONE
    petRecyclerView.visibility = View.VISIBLE
    emptyMessage.visibility = View.VISIBLE
}

private fun openChooseRoute(pet: petEntity, userEmail: String) {
    val fragment = ChooseRoute.newInstance(pet.id, pet.name, userEmail)

    parentFragmentManager.beginTransaction()
        .replace(R.id.fragment_contanier, fragment)
        .addToBackStack(null)
        .commit()
}

private fun showWalkerUI() {
    acceptedCard.visibility = View.GONE
    petRecyclerView.visibility = View.GONE
    emptyMessage.visibility = View.GONE
}

private fun loadAcceptedRequest() {
    val db = dataBaseProvider.getDatabase(requireContext())

    CoroutineScope(Dispatchers.IO).launch {
        db.walkerRequestDao().getAcceptedRequestsByWalker(userEmail)
            .collect { acceptedList ->

                withContext(Dispatchers.Main) {
                    if (acceptedList.isNotEmpty()) {
                        val req = acceptedList.first()

                        acceptedCard.visibility = View.VISIBLE
                        acceptedPetName.text = "Mascota: ${req.petName}"
                        acceptedRouteName.text = "Ruta: ${req.routeName}"

                        btnStartWalk.setOnClickListener {
                            startWalk(req)
                        }
                    } else {
                        acceptedCard.visibility = View.GONE
                    }
                }
            }
    }
}

private fun startWalk(request: walkerRequestEntity) {
    val db = dataBaseProvider.getDatabase(requireContext())
    val startTime = System.currentTimeMillis()

    CoroutineScope(Dispatchers.IO).launch {
        db.walkerRequestDao().startWalk(
            request.id,
            "En curso",
            startTime
        )

        withContext(Dispatchers.Main) {
            val fragment = WalkInProgress.newInstance(userEmail)
            parentFragmentManager.beginTransaction()
                .replace(R.id.fragment_contanier, fragment)
                .addToBackStack(null)
                .commit()
```

```
            }
        }
    }


    private fun observeWalkInProgressForOwner() {
        val db = dataBaseProvider.getDatabase(requireContext())


        lifecycleScope.launch {
            db.walkerRequestDao()
                .observeActiveWalkForOwner(userEmail)
                .collect { walk ->


                    if (walk == null) {
                        acceptedCard.visibility = View.GONE
                        return@collect
                    }


                    acceptedCard.visibility = View.VISIBLE
                    btnStartWalk.visibility = View.GONE
                    acceptedPetName.text = "Mascota: ${walk.petName}"
                    acceptedRouteName.text = "Paseo en curso"
                }
        }
    }


    private fun observeFinishedWalkForOwner() {
        val db = dataBaseProvider.getDatabase(requireContext())


        lifecycleScope.launch {
            db.walkerRequestDao()
                .observeFinishedWalkForOwner(userEmail)
                .filterNotNull()
                .take(1)
                .collect { walk ->
                    openRating(walk.walkerEmail)
                }
        }
    }


    private fun openRating(walkerEmail: String) {
        val fragment = Rating.newInstance(
            walkerEmail = walkerEmail,
            ownerEmail = userEmail
        )


        parentFragmentManager.beginTransaction()
            .replace(R.id.fragment_contanier, fragment)
            .addToBackStack(null)
            .commit()
    }
}
```

Message.kt

```
class Message : Fragment() {
    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        return inflater.inflate(R.layout.fragment_message, container, false)
    }
}
```

Pet.kt

```
class Pet : Fragment() {
    private lateinit var petRegisterTitle: TextView
    private lateinit var petName: EditText
    private lateinit var petBreed: EditText
    private lateinit var petAge: EditText
    private lateinit var petDescription: EditText
    private lateinit var petPhoto: ImageView
    private lateinit var petSelectPhoto: Button
    private lateinit var petRegister: Button
    private lateinit var pickImageLauncher: ActivityResultLauncher<String>
    private lateinit var requestTitle: TextView
    private lateinit var requestEmpty: TextView
    private lateinit var requestRecycler: RecyclerView
    private lateinit var requestAdapter: WalkerRequestAdapter
    private var selectedPhotoUri: String = ""
    private var userRole: String = "Owner"
    private lateinit var userEmail: String


    companion object {
```

```
    private const val ARG_USEREMAIL = "UserEmail"
    private const val ARG_USERROLE = "UserRole"


    fun newInstance(userEmail: String, userRole: String) : Pet {
        val fragment = Pet()
        val args = Bundle()
        args.apply {
            putString(ARG_USEREMAIL, userEmail)
            putString(ARG_USERROLE, userRole)
        }
        fragment.arguments = args
        return fragment
    }
}


override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)


    pickImageLauncher = registerForActivityResult(
        ActivityResultContracts.GetContent()
    ) {
        uri ->
        if(uri != null) {
            selectedPhotoUri = uri.toString()
            petPhoto.setImageURI(uri)
        }
    }
}


override fun onCreateView(
    inflater: LayoutInflater,
    container: ViewGroup?,
    savedInstanceState: Bundle?
): View? {
    val view = inflater.inflate(R.layout.fragment_pet, container, false)


    userEmail = arguments?.getString(ARG_USEREMAIL) ?: throw IllegalStateException("Pet Fragment requiere un userEmail válido")
    userRole = arguments?.getString(ARG_USERROLE) ?: "Owner"


    petRegisterTitle = view.findViewById(R.id.textView6)
    petName = view.findViewById(R.id.editTextName)
    petBreed = view.findViewById(R.id.editTextBreed)
    petAge = view.findViewById(R.id.editTextAge)
    petDescription = view.findViewById(R.id.editTextDescription)


    petPhoto = view.findViewById(R.id.imageDogPlaceholder)


    petSelectPhoto = view.findViewById(R.id.buttonSelectPhoto)
    petRegister = view.findViewById(R.id.buttonRegisterPet)


    petSelectPhoto.setOnClickListener {
        pickImageLauncher.launch("image/*")
    }


    petRegister.setOnClickListener { registerPet(userEmail) }


    requestTitle = view.findViewById(R.id.walkerRequestsTitle)
    requestEmpty = view.findViewById(R.id.emptyRequests)
    requestRecycler = view.findViewById(R.id.walkerRequestsRecycler)


    requestRecycler.layoutManager = LinearLayoutManager(requireContext())
    requestAdapter = WalkerRequestAdapter(
        emptyList(),
        object : WalkerRequestAdapter.OnRequestActionListener {
            override fun onAccept(request: walkerRequestEntity) {
                updateRequestStatus(request, "Aceptada")
            }


            override fun onReject(request: walkerRequestEntity) {
                updateRequestStatus(request, "Rechazada")
            }
        }
    )


    requestRecycler.adapter = requestAdapter


    loadUserRole()
```

```
    return view
}


private fun loadUserRole() {
    if(userRole == "Owner") {
        showOwnerUI()
    } else {
        showWalkerUI()
    }
}


private fun showOwnerUI() {
    petRegisterTitle.visibility = View.VISIBLE
    petName.visibility = View.VISIBLE
    petBreed.visibility = View.VISIBLE
    petAge.visibility = View.VISIBLE
    petDescription.visibility = View.VISIBLE
    petPhoto.visibility = View.VISIBLE
    petSelectPhoto.visibility = View.VISIBLE
    petRegister.visibility = View.VISIBLE


    requestTitle.visibility = View.GONE
    requestRecycler.visibility = View.GONE
}


private fun showWalkerUI() {
    petRegisterTitle.visibility = View.GONE
    petName.visibility = View.GONE
    petBreed.visibility = View.GONE
    petAge.visibility = View.GONE
    petDescription.visibility = View.GONE
    petPhoto.visibility = View.GONE
    petSelectPhoto.visibility = View.GONE
    petRegister.visibility = View.GONE


    requestTitle.visibility = View.VISIBLE
    requestRecycler.visibility = View.VISIBLE


    loadRequests()
}


private fun registerPet(userEmail: String) {
    val name = petName.text.toString()
    val breed = petBreed.text.toString()
    val age = petAge.text.toString().toInt()
    val description = petDescription.text.toString()


    if(name.isBlank() || breed.isBlank() || age.toString().isBlank() || description.isBlank() || selectedPhotoUri.isBlank()) {
        Toast.makeText(requireContext(), "Por favor, llene todos los campos", Toast.LENGTH_SHORT).show()
        return
    }


    val internalPhotoPath = saveImageToInternalStorage(selectedPhotoUri.toUri())


    val pet = petEntity(
        name = name,
        breed = breed,
        description = description,
        age = age,
        photoUri = internalPhotoPath,
        ownerEmail = userEmail
    )


    val db = dataBaseProvider.getDatabase(requireContext())


    CoroutineScope(Dispatchers.IO).launch {
        db.petDao().insertPet(pet)


        launch(Dispatchers.Main) {
            Toast.makeText(requireContext(), "Mascota registrada correctamente", Toast.LENGTH_SHORT).show()
        }
    }
}


private fun saveImageToInternalStorage(uri: Uri): String {
    val inputStream = requireContext().contentResolver.openInputStream(uri)
    val fileName = "pet_${System.currentTimeMillis()}.jpg"
```

```
        val file = File(requireContext().filesDir, fileName)
        val outputStream = FileOutputStream(file)


        inputStream?.copyTo(outputStream)
        inputStream?.close()
        outputStream.close()


        return file.absolutePath
    }


    private fun loadRequests() {
        val db = dataBaseProvider.getDatabase(requireContext())


        CoroutineScope(Dispatchers.IO).launch {
            val requests = db.walkerRequestDao().getRequestsForWalker(userEmail)
            launch(Dispatchers.Main) {
                if(requests.isNotEmpty()) {
                    requestEmpty.visibility = View.GONE
                    requestRecycler.visibility = View.VISIBLE
                    requestAdapter.updateData(requests)
                } else {
                    requestEmpty.visibility = View.VISIBLE
                    requestRecycler.visibility = View.GONE
                }
            }
        }
    }


    private fun updateRequestStatus(request: walkerRequestEntity, newStatus: String) {
        val db = dataBaseProvider.getDatabase(requireContext())


        CoroutineScope(Dispatchers.IO).launch {
            db.walkerRequestDao().updateStatus(request.id, newStatus)


            val updated = db.walkerRequestDao().getRequestsForWalker(userEmail)


            launch(Dispatchers.Main) {
                if (updated.isNotEmpty()) {
                    requestTitle.visibility = View.VISIBLE
                    requestRecycler.visibility = View.VISIBLE
                    requestAdapter.updateData(updated)
                } else {
                    requestTitle.visibility = View.GONE
                    requestRecycler.visibility = View.GONE
                    requestEmpty.visibility = View.VISIBLE
                }
            }
        }
    }
}
```

Rating.kt

```
class Rating : Fragment() {


    private lateinit var walkerEmail: String
    private lateinit var ownerEmail: String
    private val db by lazy { dataBaseProvider.getDatabase(requireContext()) }


    companion object {
        private const val ARG_WALKER_EMAIL = "walkerEmail"
        private const val ARG_OWNER_EMAIL = "ownerEmail"


        fun newInstance(walkerEmail: String, ownerEmail: String): Rating {
            return Rating().apply {
                arguments = Bundle().apply {
                    putString(ARG_WALKER_EMAIL, walkerEmail)
                    putString(ARG_OWNER_EMAIL, ownerEmail)
                }
            }
        }
    }


    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)


        walkerEmail = requireArguments().getString(ARG_WALKER_EMAIL)
            ?: error("walkerEmail requerido")


        ownerEmail = requireArguments().getString(ARG_OWNER_EMAIL)
```

```
            ?: error("ownerEmail requerido")
    }



    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        return inflater.inflate(R.layout.fragment_rating, container, false)
    }


    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {


        val ratingBar = view.findViewById<RatingBar>(R.id.ratingBar)
        val btnSubmit = view.findViewById<Button>(R.id.buttonSubmitRating)


        btnSubmit.setOnClickListener {


            val value = ratingBar.rating
            if (value == 0f) {
                Toast.makeText(requireContext(), "Selecciona una calificación", Toast.LENGTH_SHORT).show()
                return@setOnClickListener
            }


            CoroutineScope(Dispatchers.IO).launch {
                val dao = db.walkerDao()
                val existing = dao.getWalkerByEmail(walkerEmail)


                val newWalker = if (existing == null) {
                    walkerEntity(
                        walkerEmail = walkerEmail,
                        ratingSum = value,
                        ratingCount = 1,
                        ratingAverage = value
                    )
                } else {
                    val newSum = existing.ratingSum + value
                    val newCount = existing.ratingCount + 1
                    val newAvg = newSum / newCount


                    existing.copy(
                        ratingSum = newSum,
                        ratingCount = newCount,
                        ratingAverage = newAvg
                    )
                }


                dao.insertWalker(newWalker)


                db.walkerRequestDao()
                    .markWalkAsRated(ownerEmail)


                withContext(Dispatchers.Main) {
                    Toast.makeText(requireContext(), "¡Gracias por tu calificación!", Toast.LENGTH_SHORT).show()
                    parentFragmentManager.popBackStack()
                }
            }
        }
    }
}
```

Route.kt

```
class Route : Fragment() {


    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        return inflater.inflate(R.layout.fragment_route, container, false)
    }


    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)


        val recycler = view.findViewById<RecyclerView>(R.id.routesRecycler)
        recycler.layoutManager = LinearLayoutManager(requireContext())
```

```
        recycler.adapter = RouteAdapter(PredefinedRoutes.allRoutes) { selectedRoute ->
            openMap(selectedRoute)
        }
    }

    private fun openMap(route: PredefinedRoute) {
        val fragment = RouteMapFragment()

        val bundle = Bundle()
        bundle.putSerializable("route", route)
        fragment.arguments = bundle

        parentFragmentManager.beginTransaction()
            .replace(R.id.fragment_contanier, fragment)
            .addToBackStack(null)
            .commit()
    }
}
```

RouteMapFragment.kt

```
class RouteMapFragment : Fragment(), OnMapReadyCallback {

    private lateinit var selectedRoute: PredefinedRoute

    private var pet: com.example.animalcrossing.data.entity.petEntity? = null
    private var walkerUser: com.example.animalcrossing.data.entity.userEntity? = null
    private var walkerData: com.example.animalcrossing.data.entity.walkerEntity? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        selectedRoute =
            requireArguments().getSerializable("route") as PredefinedRoute

        pet =
            requireArguments().getSerializable("pet") as? com.example.animalcrossing.data.entity.petEntity
        walkerUser =
            requireArguments().getSerializable("walkerUser") as? com.example.animalcrossing.data.entity.userEntity
        walkerData =
            requireArguments().getSerializable("walkerData") as? com.example.animalcrossing.data.entity.walkerEntity
    }

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        return inflater.inflate(R.layout.fragment_route_map, container, false)
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        val mapFragment =
            childFragmentManager.findFragmentById(R.id.mapFragment) as SupportMapFragment
        mapFragment.getMapAsync(this)

        val infoCard = view.findViewById<View>(R.id.infoCard)
        val previewText = view.findViewById<TextView>(R.id.previewText)

        val petNameView = view.findViewById<TextView>(R.id.mapPetName)
        val routeNameView = view.findViewById<TextView>(R.id.mapRouteName)
        val walkerNameView = view.findViewById<TextView>(R.id.mapWalkerName)
        val finishBtn = view.findViewById<Button>(R.id.buttonFinishWalk)

        routeNameView.text = selectedRoute.name

        if (pet == null || walkerUser == null) {
            infoCard.visibility = View.GONE
            previewText.visibility = View.VISIBLE
            return
        }

        infoCard.visibility = View.VISIBLE
        previewText.visibility = View.GONE
```

```
        petNameView.text = pet?.name ?: ""
        walkerNameView.text = walkerUser?.name ?: ""


        finishBtn.setOnClickListener {
            val frag = Rating()
            val b = Bundle()
            b.putString("walkerEmail", walkerUser?.userEmail)
            frag.arguments = b


            parentFragmentManager.beginTransaction()
                .replace(R.id.fragment_contanier, frag)
                .addToBackStack(null)
                .commit()
        }
    }


    override fun onMapReady(googleMap: GoogleMap) {


        googleMap.addPolyline(
            PolylineOptions().apply {
                addAll(selectedRoute.points.map { LatLng(it.latitude, it.longitude) })
                width(12f)
            }
        )


        val bounds = LatLngBounds.builder()
        selectedRoute.points.forEach { p ->
            bounds.include(LatLng(p.latitude, p.longitude))
        }


        googleMap.animateCamera(
            CameraUpdateFactory.newLatLngBounds(bounds.build(), 150)
        )
    }
}
```

WalkerList.kt

```
class WalkerList : Fragment() {


    companion object {
        private const val ARG_ROUTE = "selectedRoute"
        private const val ARG_PET_ID = "petId"
        private const val ARG_PET_NAME = "petName"
        private const val ARG_USER = "userData"


        fun newInstance(
            route: PredefinedRoute,
            petId: Int,
            petName: String,
            user: userEntity
        ): WalkerList {
            val fragment = WalkerList()
            val args = Bundle().apply {
                putSerializable(ARG_ROUTE, route)
                putInt(ARG_PET_ID, petId)
                putString(ARG_PET_NAME, petName)
                putSerializable(ARG_USER, user)
            }
            fragment.arguments = args
            return fragment
        }
    }


    private lateinit var recyclerView: RecyclerView
    private lateinit var adapter: WalkerAdapter
    private lateinit var emptyMessage: TextView


    private lateinit var selectedRoute: PredefinedRoute
    private var petId: Int = -1
    private var petName: String = ""
    private lateinit var user: userEntity


    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        val view = inflater.inflate(R.layout.fragment_walker_list, container, false)
```

```
        recyclerView = view.findViewById(R.id.walkersRecycler)
        emptyMessage = view.findViewById(R.id.emptyWalkerMessage)


        recyclerView.layoutManager = LinearLayoutManager(requireContext())


        selectedRoute = requireArguments().getSerializable(ARG_ROUTE) as PredefinedRoute
        petId = requireArguments().getInt(ARG_PET_ID)
        petName = requireArguments().getString(ARG_PET_NAME) ?: ""
        user = requireArguments().getSerializable(ARG_USER) as userEntity


        adapter = WalkerAdapter(emptyList()) { walker ->
            openWalkerProfile(walker)
        }


        recyclerView.adapter = adapter


        loadWalkers()


        return view
    }


    private fun loadWalkers() {
        val db = dataBaseProvider.getDatabase(requireContext())


        viewLifecycleOwner.lifecycleScope.launch {
            db.walkerDao()
                .getWalkersWithRating()
                .collect { walkers ->


                    adapter.updateData(walkers)


                    if (walkers.isEmpty()) {
                        emptyMessage.visibility = View.VISIBLE
                        recyclerView.visibility = View.GONE
                    } else {
                        emptyMessage.visibility = View.GONE
                        recyclerView.visibility = View.VISIBLE
                    }
                }
        }
    }


    private fun openWalkerProfile(walker: WalkerWithRating) {
        val fragment = WalkerProfile.newInstance(
            walkerEmail = walker.email,
            route = selectedRoute,
            petId = petId,
            petName = petName,
            user = user
        )


        parentFragmentManager.beginTransaction()
            .replace(R.id.fragment_contanier, fragment)
            .addToBackStack(null)
            .commit()
    }
}
```

WalkerProfile.kt

```
class WalkerProfile : Fragment() {


    companion object {
        private const val ARG_WALKER = "walker"
        private const val ARG_ROUTE = "route"
        private const val ARG_PET_ID = "petId"
        private const val ARG_PET_NAME = "petName"
        private const val ARG_USER = "user"
        private const val ARG_OWNER = "owner"


        fun newInstance(
            walkerEmail: String,
            route: PredefinedRoute,
            petId: Int,
            petName: String,
            user: userEntity
        ): WalkerProfile {
            val fragment = WalkerProfile()
            val args = Bundle().apply {
                putString(ARG_WALKER, walkerEmail)
```

```
            putSerializable(ARG_ROUTE, route)
            putInt(ARG_PET_ID, petId)
            putString(ARG_PET_NAME, petName)
            putSerializable(ARG_USER, user)
        }
        fragment.arguments = args
        return fragment
    }
}


private lateinit var walkerEmailValue: String
private lateinit var route: PredefinedRoute
private lateinit var owner: userEntity
private var petId: Int = -1
private var petName: String = ""


private lateinit var walkerName: TextView
private lateinit var walkerEmail: TextView
private lateinit var routeName: TextView
private lateinit var routeTime: TextView
private lateinit var routePrice: TextView
private lateinit var confirmButton: Button


override fun onCreateView(
    inflater: LayoutInflater,
    container: ViewGroup?,
    savedInstanceState: Bundle?
): View {


    val view = inflater.inflate(R.layout.fragment_walker_profile, container, false)


    walkerEmailValue = requireArguments().getString(ARG_WALKER)
        ?: error("walkerEmail requerido")


    route = requireArguments().getSerializable(ARG_ROUTE) as PredefinedRoute
    owner = requireArguments().getSerializable(ARG_USER) as userEntity
    petId = requireArguments().getInt(ARG_PET_ID)
    petName = requireArguments().getString(ARG_PET_NAME) ?: ""


    walkerName = view.findViewById(R.id.profileWalkerName)
    walkerEmail = view.findViewById(R.id.profileWalkerEmail)
    routeName = view.findViewById(R.id.profileRouteName)
    routeTime = view.findViewById(R.id.profileRouteTime)
    routePrice = view.findViewById(R.id.profileRoutePrice)


    loadWalkerInfo()


    routeName.text = route.name
    routeTime.text = route.time
    routePrice.text = route.price


    confirmButton = view.findViewById(R.id.buttonConfirmWalk)
    confirmButton.setOnClickListener {
        createWalkRequest()
    }


    return view
}



private fun loadWalkerInfo() {
    val db = dataBaseProvider.getDatabase(requireContext())


    lifecycleScope.launch(Dispatchers.IO) {


        val walkerUser = db.userDao().getUserById(walkerEmailValue)
        val walkerRating = db.walkerDao().getWalkerByEmail(walkerEmailValue)


        val name = walkerUser?.name ?: "Paseador"
        val email = walkerUser?.userEmail ?: walkerEmailValue


        val ratingText = walkerRating?.ratingAverage?.let {
            "⭐ %.1f".format(it)
        } ?: "Sin calificaciones"
```

```
            withContext(Dispatchers.Main) {
                walkerName.text = name
                walkerEmail.text = email
                view?.findViewById<TextView>(R.id.walkerRating)?.text = ratingText
            }
        }
    }


    private fun createWalkRequest() {
        val db = dataBaseProvider.getDatabase(requireContext())


        val request = walkerRequestEntity(
            walkerEmail = walkerEmailValue,
            ownerEmail = owner.userEmail,
            petId = petId,
            petName = petName,
            routeName = route.name,
            status = "Pendiente"
        )


        CoroutineScope(Dispatchers.IO).launch {
            db.walkerRequestDao().insertRequest(request)


            launch(Dispatchers.Main) {
                Toast.makeText(
                    requireContext(),
                    "¡Solicitud enviada correctamente!",
                    Toast.LENGTH_LONG
                ).show()


                parentFragmentManager.popBackStack()
            }
        }
    }
}
```

WalkInProgress.kt

```
class WalkInProgress : Fragment() {
    private lateinit var textPet: TextView
    private lateinit var textRoute: TextView
    private lateinit var textTime: TextView
    private lateinit var imageEvidence: ImageView
    private lateinit var btnUploadPhoto: Button
    private lateinit var btnFinish: Button


    private lateinit var userEmail: String


    private var selectedImageUri: Uri? = null


    private val imagePicker =
        registerForActivityResult(ActivityResultContracts.GetContent()) { uri ->
            uri?.let {
                selectedImageUri = it
                imageEvidence.setImageURI(it)
            }
        }




    companion object {
        private const val ARG_USER_EMAIL = "UserEmail"


        fun newInstance(userEmail: String): WalkInProgress {
            val fragment = WalkInProgress()
            val args = Bundle()
            args.putString(ARG_USER_EMAIL, userEmail)
            fragment.arguments = args
            return fragment
        }
    }


    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        val view = inflater.inflate(R.layout.fragment_walk_in_progress, container, false)


        userEmail = requireArguments().getString(ARG_USER_EMAIL) ?: error("UserEmail requerido")
```

```
        textPet = view.findViewById(R.id.walkPetName)
        textRoute = view.findViewById(R.id.walkRouteName)
        textTime = view.findViewById(R.id.walkTime)
        imageEvidence = view.findViewById(R.id.evidenceImage)
        btnUploadPhoto = view.findViewById(R.id.uploadPhotoButton)


        btnUploadPhoto.setOnClickListener {
            imagePicker.launch("image/*")
        }


        btnFinish = view.findViewById(R.id.finishWalkButton)


        observeWalk()


        return view
    }


    private fun observeWalk() {
        val db = dataBaseProvider.getDatabase(requireContext())


        lifecycleScope.launch {
            db.walkerRequestDao()
                .observeActiveWalkForWalker(userEmail)
                .collect { walk ->


                    if (walk == null) return@collect


                    textPet.text = "Mascota: ${walk.petName}"
                    textRoute.text = "Ruta: ${walk.routeName}"


                    if (walk.startTime != null) {
                        startTimer(walk.startTime)
                    }


                    btnFinish.setOnClickListener {
                        if (selectedImageUri == null) {
                            Toast.makeText(requireContext(), "Debes subir una foto como evidencia", Toast.LENGTH_SHORT).show()
                            return@setOnClickListener
                        }


                        finishWalk(walk.id)
                    }
                }
        }
    }


    private fun startTimer(startTime: Long) {
        lifecycleScope.launch {
            while (true) {
                val elapsed = System.currentTimeMillis() - startTime
                textTime.text = formatTime(elapsed)
                delay(1000)
            }
        }
    }


    private fun formatTime(ms: Long): String {
        val seconds = (ms / 1000) % 60
        val minutes = (ms / 1000 / 60) % 60
        val hours = ms / 1000 / 60 / 60


        return String.format("%02d:%02d:%02d", hours, minutes, seconds)
    }


    private fun finishWalk(id: Int) {
        val db = dataBaseProvider.getDatabase(requireContext())


        lifecycleScope.launch(Dispatchers.IO) {
            db.walkerRequestDao().updateStatus(id, "Finalizado")
        }


        parentFragmentManager.popBackStack()
    }
}
```

AcceptedRequestAdapter.kt

```
class AcceptedRequestAdapter(
    private var requests: List<walkerRequestEntity>,
    private val onSelect: (walkerRequestEntity) -> Unit
) : RecyclerView.Adapter<AcceptedRequestAdapter.ViewHolder>() {


    inner class ViewHolder(view: View) : RecyclerView.ViewHolder(view) {
        val petName = view.findViewById<TextView>(R.id.petName)
        val btnStartWalk = view.findViewById<Button>(R.id.startWalkButton)
    }


    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ViewHolder {
        val v = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_accepted_requests, parent, false)
        return ViewHolder(v)
    }


    override fun onBindViewHolder(holder: ViewHolder, position: Int) {
        val request = requests[position]


        holder.petName.text = request.petName


        holder.btnStartWalk.setOnClickListener {
            onSelect(request)
        }
    }


    override fun getItemCount() = requests.size


    fun updateData(newData: List<walkerRequestEntity>) {
        requests = newData
        notifyDataSetChanged()
    }
}
```

PetAdapter.kt

```
class PetAdapter(
    private var pets: List<petEntity>,
    private val onPetSelected: ((petEntity) -> Unit)? = null
) : RecyclerView.Adapter<PetAdapter.PetViewHolder>() {


    inner class PetViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
        val petImage: ImageView = itemView.findViewById(R.id.petImage)
        val petName: TextView = itemView.findViewById(R.id.petName)
        val petBreed: TextView = itemView.findViewById(R.id.petBreed)
    }


    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): PetViewHolder {
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_pet_card, parent, false)
        return PetViewHolder(view)
    }


    override fun onBindViewHolder(holder: PetViewHolder, position: Int) {
        val pet = pets[position]


        holder.petName.text = pet.name
        holder.petBreed.text = pet.breed
        holder.petImage.setImageURI(Uri.parse(pet.photoUri))


        holder.itemView.setOnClickListener {
            onPetSelected?.invoke(pet)
        }
    }


    override fun getItemCount(): Int = pets.size


    fun updateData(newPets: List<petEntity>) {
        pets = newPets
        notifyDataSetChanged()
    }
}
```

RouteAdapter.kt

```
class RouteAdapter (
    private val routes: List<PredefinedRoute>,
    private val onClick: (PredefinedRoute) -> Unit
```

```
) : RecyclerView.Adapter<RouteAdapter.RouteViewHolder>() {


    inner class RouteViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
        val nameRoute = itemView.findViewById<TextView>(R.id.routeName)
        val price = itemView.findViewById<TextView>(R.id.routePrice)
        val timeDistance = itemView.findViewById<TextView>(R.id.routeTimeDistance)
    }


    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): RouteViewHolder {
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_route_card, parent, false)
        return RouteViewHolder(view)
    }


    override fun onBindViewHolder(holder: RouteViewHolder, position: Int) {
        val route = routes[position]


        holder.nameRoute.text = route.name
        holder.price.text = route.price
        holder.timeDistance.text = "${route.time} - ${route.distance}"


        holder.itemView.setOnClickListener {
            onClick(route)
        }
    }


    override fun getItemCount() = routes.size
}
```

WalkerAdapter.kt

```
class WalkerAdapter(
    private var walkerList: List<WalkerWithRating>,
    private val onClick: (WalkerWithRating) -> Unit
) : RecyclerView.Adapter<WalkerAdapter.WalkerViewHolder>() {


    inner class WalkerViewHolder(view: View) : RecyclerView.ViewHolder(view) {
        val walkerName: TextView = view.findViewById(R.id.walkerName)
        val walkerEmail: TextView = view.findViewById(R.id.walkerEmail)
        val walkerRating: TextView = view.findViewById(R.id.walkerRating)
    }


    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): WalkerViewHolder {
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_walker_card, parent, false)
        return WalkerViewHolder(view)
    }


    override fun onBindViewHolder(holder: WalkerViewHolder, position: Int) {
        val walker = walkerList[position]


        holder.walkerName.text = walker.name
        holder.walkerEmail.text = walker.email


        holder.walkerRating.text =
            if (walker.ratingAverage != null)
                "⭐ ${String.format("%.1f", walker.ratingAverage)}"
            else
                "⭐ Sin calificaciones"


        holder.itemView.setOnClickListener { onClick(walker) }
    }


    override fun getItemCount(): Int = walkerList.size


    fun updateData(newList: List<WalkerWithRating>) {
        walkerList = newList
        notifyDataSetChanged()
    }
}
```

WalkerRequestAdapter.kt

```
class WalkerRequestAdapter(
    private var requests: List<walkerRequestEntity>,
    private val listener: OnRequestActionListener
) : RecyclerView.Adapter<WalkerRequestAdapter.RequestViewHolder>() {


    interface OnRequestActionListener {
```

```
        fun onAccept(request: walkerRequestEntity)
        fun onReject(request: walkerRequestEntity)
    }


    fun updateData(newList: List<walkerRequestEntity>) {
        requests = newList
        notifyDataSetChanged()
    }


    inner class RequestViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
        val textOwner: TextView = itemView.findViewById(R.id.textOwner)
        val textPet: TextView = itemView.findViewById(R.id.textPet)
        val textRoute: TextView = itemView.findViewById(R.id.textRoute)
        val textStatus: TextView = itemView.findViewById(R.id.textStatus)
        val buttonAccept: Button = itemView.findViewById(R.id.buttonAccept)
        val buttonReject: Button = itemView.findViewById(R.id.buttonReject)
    }


    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): RequestViewHolder {
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_walk_request, parent, false)
        return RequestViewHolder(view)
    }


    override fun onBindViewHolder(holder: RequestViewHolder, position: Int) {
        val request = requests[position]


        holder.textOwner.text = "Dueño: ${request.ownerEmail}"
        holder.textPet.text = "Mascota: ${request.petName}"
        holder.textRoute.text = "Ruta: ${request.routeName}"
        holder.textStatus.text = "Estado: ${request.status}"


        val isPending = request.status == "Pendiente"
        holder.buttonAccept.isEnabled = isPending
        holder.buttonReject.isEnabled = isPending


        holder.buttonAccept.setOnClickListener {
            listener.onAccept(request)
        }


        holder.buttonReject.setOnClickListener {
            listener.onReject(request)
        }
    }


    override fun getItemCount(): Int = requests.size
}
```

UserWithWalkerData.kt

```
data class UserWithWalkerData (
    val user: com.example.animalcrossing.data.entity.userEntity?,
    val walker: com.example.animalcrossing.data.entity.walkerEntity?
) : java.io.Serializable
```

WalkerWithRating.kt

```
data class WalkerWithRating(
    val name: String,
    val email: String,
    val ratingAverage: Float?
)
```

AppDatabase.kt

```
@Database(
    entities = [petEntity::class, userEntity::class, walkerEntity::class, walkerRequestEntity::class],
    version = 3,
    exportSchema = false
)


abstract class AppDatabase : RoomDatabase() {
    abstract fun userDao(): userDao
    abstract fun petDao(): petDao
    abstract fun walkerDao(): walkerDao
    abstract fun walkerRequestDao(): walkerRequestDao
}
```

DataBaseProvider.kt

```
object dataBaseProvider {
    @Volatile
    private var INSTANCE: AppDatabase? = null
```

```
    fun getDatabase(context: Context): AppDatabase {
        return INSTANCE ?: synchronized(this) {
            val instance = Room.databaseBuilder(
                context.applicationContext,
                AppDatabase::class.java,
                "animalCrossingDB"
            )
            .fallbackToDestructiveMigration(dropAllTables = true)
            .fallbackToDestructiveMigrationOnDowngrade(dropAllTables = true)
            .build()


            INSTANCE = instance
            instance
        }
    }
}
```

PredefinedRoutes.kt

```
object PredefinedRoutes {
    val shortRoute = PredefinedRoute(
        name = "Ruta Corta",
        time = "10 minutos",
        price = "$40 MXN",
        distance = "600 m",
        points = listOf(
            RoutePoint(19.396329, -99.095250),
            RoutePoint(19.394700, -99.095454),
            RoutePoint(19.394821, -99.096629),
            RoutePoint(19.396440, -99.096452),
            RoutePoint(19.396329, -99.095250)
        )
    )


    val longRoute = PredefinedRoute(
        name = "Ruta Larga",
        time = "20 minutos",
        price = "$75 MXN",
        distance = "1.2 km",
        points = listOf(
            RoutePoint(19.396329, -99.095250),
            RoutePoint(19.394700, -99.095454),
            RoutePoint(19.394993, -99.098694),
            RoutePoint(19.396577, -99.098335),
            RoutePoint(19.396329, -99.095250)
        ),
    )


    val allRoutes = listOf(shortRoute, longRoute)
}


data class RoutePoint(
    val latitude: Double,
    val longitude: Double
)


data class PredefinedRoute (
    val name: String,
    val time: String,
    val price: String,
    val distance: String,
    val points: List<RoutePoint>,
) : Serializable
```

conversationEntity.kt

```
@Entity(tableName = "conversations",
    foreignKeys = [
        ForeignKey(
            entity = userEntity::class,
            parentColumns = ["userEmail"],
            childColumns = ["ownerEmail"],
            onDelete = ForeignKey.CASCADE
        ),
        ForeignKey(
            entity = walkerEntity::class,
            parentColumns = ["walkerEmail"],
            childColumns = ["walkerEmail"],
            onDelete = ForeignKey.CASCADE
        )
    ])
data class conversationEntity (
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val ownerEmail: String,
    val walkerEmail: String,
    val date: Long
)
```

messageEntity.kt

```
@Entity(tableName = "messages",
    foreignKeys = [
        ForeignKey(
            entity = userEntity::class,
            parentColumns = ["userEmail"],
            childColumns = ["userEmisor"]
        ),
        ForeignKey(
            entity = walkerEntity::class,
            parentColumns = ["walkerEmail"],
            childColumns = ["walkerReciever"]
        )
    ])
data class messageEntity (
    @PrimaryKey(autoGenerate = true) val id: Int,
    val userEmisor: String,
    val walkerReciever: String,
    val message: String,
    val date: Long,
    val isRead: Boolean
)
```

petEntity.kt

```
@Entity(tableName = "pets",
    foreignKeys = [
        ForeignKey(
            entity = userEntity::class,
            parentColumns = ["userEmail"],
            childColumns = ["ownerEmail"],
            onDelete = ForeignKey.CASCADE
        )
    ]
)
data class petEntity (
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val name: String,
    val breed: String,
    val description: String,
    val photoUri: String,
    val age: Int,
    val ownerEmail: String
) : Serializable
```

routeEntity.kt

```
@Entity(tableName = "routes")
data class routeEntity (
    @PrimaryKey(autoGenerate = true) val id: Int,
    val latitude: Double,
    val longitude: Double,
    val distance: Float,
    val duration: Float
)
```

userEntity.kt

```
@Entity(tableName = "users")
data class userEntity (
    @PrimaryKey(autoGenerate = false) val userEmail: String,
    val password: String,
    val name: String,
    val address: String,
    val telephoneNumber: String,
    val role: String
) : Serializable
```

walkerEntity.kt

```
@Entity(tableName = "walkers",
    foreignKeys = [
        ForeignKey(
            entity = userEntity::class,
            parentColumns = ["userEmail"],
            childColumns = ["walkerEmail"],
            onDelete = ForeignKey.CASCADE
        )
    ])
data class walkerEntity (
    @PrimaryKey(autoGenerate = false) val walkerEmail: String,
    val ratingSum: Float = 0f,
    val ratingCount: Int = 0,
    val ratingAverage: Float = 0f
) : Serializable
```

walkerRequestEntity.kt

```
@Entity(
    tableName = "walkerRequests",
    foreignKeys = [
        ForeignKey(
            entity = userEntity::class,
            parentColumns = ["userEmail"],
            childColumns = ["ownerEmail"],
            onDelete = ForeignKey.CASCADE
        ),
```

```
        ForeignKey(
            entity = userEntity::class,
            parentColumns = ["userEmail"],
            childColumns = ["walkerEmail"],
            onDelete = ForeignKey.CASCADE
        ),
        ForeignKey(
            entity = petEntity::class,
            parentColumns = ["id"],
            childColumns = ["petId"],
            onDelete = ForeignKey.CASCADE
        )
    ]
)
data class walkerRequestEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val walkerEmail: String,
    val ownerEmail: String,
    val petId: Int,
    val petName: String,
    val routeName: String,
    val status: String,      // Pendiente, Aceptada, EnCurso, Completada


    val startTime: Long? = null,
    val currentLat: Double? = null,
    val currentLang: Double? = null,
    val distanceWalked: Float? = 0f,


    val rated: Boolean = false
)
```

petDao.kt

```
@Dao
interface petDao {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertPet(pet: petEntity)


    @Delete
    suspend fun deletePet(pet: petEntity)


    @Query("SELECT * FROM pets")
    fun getAllPets(): Flow<List<petEntity>>


    @Query("SELECT * FROM pets WHERE ownerEmail = :email")
    fun getPetsByOwner(email: String): Flow<List<petEntity>>
}
```

userDao.kt

```
@Dao
interface userDao {
    @Insert
    suspend fun insertUser(user: userEntity)


    @Delete
    suspend fun deleteUser(user: userEntity)


    @Query("SELECT * FROM users WHERE userEmail = :email")
    fun getUserById(email: String): userEntity?


    @Query("UPDATE users SET password = :newPassword, name = :newName, address = :newAddress, telephoneNumber = :newTelephoneNumber WHERE userEmail
= :email")
    fun updateUserById(email: String, newPassword: String, newName: String, newAddress: String, newTelephoneNumber: String): Int


    @Query("SELECT * FROM users where role = 'Walker'")
    suspend fun getWalkerUsers(): List<userEntity>
}
```

walkerDao.kt

```
@Dao
interface walkerDao {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertWalker(walker: walkerEntity)


    @Query("SELECT * FROM walkers WHERE walkerEmail = :email")
    suspend fun getWalkerByEmail(email: String): walkerEntity?


    @Query("SELECT * FROM walkers")
    fun getAllWalkers(): List<walkerEntity>


    @Query("SELECT u.name AS name, u.userEmail AS email, w.ratingAverage AS ratingAverage FROM users u LEFT JOIN walkers w ON u.userEmail = w.walkerEmail
```

```
  WHERE u.role = 'Walker'")
    fun getWalkersWithRating(): Flow<List<WalkerWithRating>>
}
```

walkerRequestDato.kt

```
@Dao
interface walkerRequestDao {


    @Insert
    suspend fun insertRequest(request: walkerRequestEntity)


    @Query("SELECT * FROM walkerRequests WHERE ownerEmail = :email")
    fun getRequestsByOwner(email: String): List<walkerRequestEntity>


    @Query("SELECT * FROM walkerRequests WHERE walkerEmail = :walkerEmail")
    suspend fun getRequestsForWalker(walkerEmail: String): List<walkerRequestEntity>


    @Query("UPDATE walkerRequests SET status = :newStatus WHERE id = :id")
    suspend fun updateStatus(id: Int, newStatus: String)


    @Query("SELECT * FROM walkerRequests WHERE walkerEmail = :walkerEmail AND status = 'Pendiente'")
    fun getPendingRequestsByWalker(walkerEmail: String): Flow<List<walkerRequestEntity>>


    @Query("SELECT * FROM walkerRequests WHERE walkerEmail = :walkerEmail AND status = 'Aceptada'")
    fun getAcceptedRequestsByWalker(walkerEmail: String): Flow<List<walkerRequestEntity>>


    @Query(" SELECT * FROM walkerRequests WHERE ownerEmail = :ownerEmail AND status = 'En curso' LIMIT 1")
    fun observeActiveWalkForOwner(ownerEmail: String): Flow<walkerRequestEntity?>


    @Query(" SELECT * FROM walkerRequests WHERE walkerEmail = :walkerEmail AND status = 'En curso' LIMIT 1")
    fun observeActiveWalkForWalker(walkerEmail: String): Flow<walkerRequestEntity?>


    @Query(" UPDATE walkerRequests SET status = :status, startTime = :startTime WHERE id = :id")
    suspend fun startWalk(id: Int, status: String, startTime: Long)


    @Query(" SELECT * FROM walkerRequests WHERE ownerEmail = :ownerEmail AND status = 'Finalizado' AND rated = 0 LIMIT 1")
    fun observeFinishedWalkForOwner(ownerEmail: String): Flow<walkerRequestEntity?>


    @Query("UPDATE walkerRequests SET rated = 1 WHERE id = :id")
    suspend fun markAsRated(id: Int)


    @Query("UPDATE walkerRequests SET rated = 1 WHERE ownerEmail = :ownerEmail AND status = 'Finalizado'")
    suspend fun markWalkAsRated(ownerEmail: String)
}
```

Para los layouts se hará algo similar, ya que la cabecera siempre será la misma: *<?xml version="1.0" encoding="utf-8"?>*:

fragment_account.xml

```
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/frameLayout4"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="12dp"
    tools:context=".Account">


    <TextView
        android:id="@+id/titleChangeData"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/title_account_change"
        android:textSize="24sp"
        android:textStyle="bold"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />


    <EditText
        android:id="@+id/userEmailReadOnly"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginTop="24dp"
        android:enabled="false"
```

```
    android:hint="@string/email"
    android:inputType="textEmailAddress"
    app:layout_constraintTop_toBottomOf="@id/titleChangeData"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />


<EditText
    android:id="@+id/editTextUserName"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:hint="@string/petName"
    android:inputType="textPersonName"
    app:layout_constraintTop_toBottomOf="@id/userEmailReadOnly"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />


<EditText
    android:id="@+id/editTextUserPassword"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:hint="@string/password"
    android:inputType="textPassword"
    app:layout_constraintTop_toBottomOf="@id/editTextUserName"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />


<EditText
    android:id="@+id/editTextUserAddress"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:hint="@string/address"
    app:layout_constraintTop_toBottomOf="@id/editTextUserPassword"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />


<EditText
    android:id="@+id/editTextUserTelephone"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:hint="@string/number"
    android:inputType="phone"
    android:maxLength="8"
    app:layout_constraintTop_toBottomOf="@id/editTextUserAddress"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />


<Button
    android:id="@+id/buttonSaveChanges"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="20dp"
    android:text="@string/saveChangesButton"
    android:background="@color/darkGreen"
    android:textColor="@color/background"
    app:layout_constraintTop_toBottomOf="@id/editTextUserTelephone"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />


<TextView
    android:id="@+id/titleLogout"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/title_account_close"
    android:textSize="24sp"
    android:textStyle="bold"
    android:layout_marginTop="32dp"
    app:layout_constraintTop_toBottomOf="@id/buttonSaveChanges"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />


<Button
    android:id="@+id/buttonLogout"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:text="@string/title_account_close"
    android:background="@color/darkGreen"
    android:textColor="@color/background"
    app:layout_constraintTop_toBottomOf="@id/titleLogout"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />
```

```
</androidx.constraintlayout.widget.ConstraintLayout>
```

fragment_choose_route.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">


    <TextView
        android:id="@+id/title"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Selecciona una ruta"
        android:textSize="20sp"
        android:textStyle="bold"
        android:paddingBottom="12dp" />


    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/routeRecycler"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />


</LinearLayout>
```

fragment_home.kt

```
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/frameLayout2"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".Home">


    <TextView
        android:id="@+id/userGreeting"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textColor="@color/black"
        android:textSize="20sp"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        android:layout_margin="16dp" />


    <TextView
        android:id="@+id/userQuestion"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/userQuestion"
        android:textColor="@color/black"
        android:textSize="16sp"
        app:layout_constraintTop_toBottomOf="@+id/userGreeting"
        app:layout_constraintStart_toStartOf="parent"
        android:layout_marginStart="16dp" />


    <androidx.cardview.widget.CardView
        android:id="@+id/acceptedRequestCard"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        app:cardElevation="6dp"
        app:cardCornerRadius="12dp"
        android:layout_margin="16dp"
        app:layout_constraintTop_toBottomOf="@+id/userQuestion"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent">


        <LinearLayout
            android:orientation="vertical"
            android:padding="16dp"
            android:layout_width="match_parent"
            android:layout_height="wrap_content">


            <TextView
                android:id="@+id/acceptedRequestPetName"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Mascota: —"
                android:textSize="18sp"
                android:textStyle="bold" />


            <TextView
```

```
            android:id="@+id/acceptedRequestRouteName"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Ruta: —"
            android:textSize="16sp"
            android:layout_marginTop="4dp" />


        <Button
            android:id="@+id/startWalkButton"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Iniciar paseo"
            android:layout_marginTop="12dp" />
    </LinearLayout>


</androidx.cardview.widget.CardView>


<TextView
    android:id="@+id/emptyMessage"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="No tienes mascotas registradas todavía."
    android:textSize="16sp"
    android:textColor="@color/black"
    android:visibility="gone"
    app:layout_constraintTop_toBottomOf="@id/acceptedRequestCard"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    android:layout_marginTop="16dp" />


<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/petRecycleViewer"
    android:layout_width="0dp"
    android:layout_height="0dp"
    android:padding="12dp"
    app:layout_constraintTop_toBottomOf="@+id/acceptedRequestCard"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    tools:listitem="@layout/item_pet_card" />


</androidx.constraintlayout.widget.ConstraintLayout>
```

fragment_message.xml

```
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:theme="@style/Theme.AnimalCrossing"
    tools:context=".Message">


</FrameLayout>
```

fragment_pet.xml

```
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/frameLayout"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/background"
    tools:theme="@style/Theme.AnimalCrossing"
    tools:context=".Pet">


    <TextView
        android:id="@+id/walkerRequestsTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Solicitudes de paseo"
        android:textSize="20sp"
        android:textStyle="bold"
        android:visibility="gone"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />


    <TextView
        android:id="@+id/emptyRequests"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="No tienes solicitudes de paseo"
        android:textSize="18sp"
        android:textStyle="bold"
        android:visibility="gone"
```

```
    app:layout_constraintTop_toBottomOf="@id/walkerRequestsTitle"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    android:layout_marginTop="12dp"/>


<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/walkerRequestsRecycler"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:visibility="gone"
    android:padding="8dp"
    app:layout_constraintTop_toBottomOf="@id/walkerRequestsTitle"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />


<ImageView
    android:id="@+id/imageDogPlaceholder"
    android:layout_width="130dp"
    android:layout_height="130dp"
    android:layout_marginTop="96dp"
    android:src="@drawable/dog_placeholder"
    app:layout_constraintBottom_toTopOf="@+id/editTextName"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.498"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.0" />


<EditText
    android:id="@+id/editTextName"
    android:layout_width="355dp"
    android:layout_height="48dp"
    android:ems="10"
    android:hint="@string/petName"
    android:inputType="text"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.417" />


<EditText
    android:id="@+id/editTextDescription"
    android:layout_width="354dp"
    android:layout_height="123dp"
    android:layout_marginTop="152dp"
    android:ems="10"
    android:hint="@string/petDescription"
    android:inputType="text"
    android:textAlignment="viewStart"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.491"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/imageDogPlaceholder"
    app:layout_constraintVertical_bias="0.2" />


<EditText
    android:id="@+id/editTextAge"
    android:layout_width="95dp"
    android:layout_height="48dp"
    android:layout_marginTop="20dp"
    android:ems="10"
    android:hint="@string/petAge"
    android:inputType="number"
    android:maxLength="2"
    app:layout_constraintBottom_toTopOf="@+id/editTextDescription"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.911"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/editTextName"
    app:layout_constraintVertical_bias="0.043" />


<EditText
    android:id="@+id/editTextBreed"
    android:layout_width="247dp"
    android:layout_height="48dp"
    android:layout_marginStart="28dp"
    android:layout_marginTop="20dp"
    android:ems="10"
    android:hint="@string/petBreed"
    android:inputType="text"
    app:layout_constraintBottom_toTopOf="@+id/editTextDescription"
    app:layout_constraintEnd_toStartOf="@+id/editTextAge"
    app:layout_constraintHorizontal_bias="0.0"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/editTextName"
```

```
        app:layout_constraintVertical_bias="0.043" />


    <Button
        android:id="@+id/buttonRegisterPet"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/registerPet"
        android:background="@color/darkGreen"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.498"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/editTextDescription"
        app:layout_constraintVertical_bias="0.205" />


    <TextView
        android:id="@+id/textView6"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/title_pet_fragment"
        android:textAlignment="center"
        android:textSize="24sp"
        android:textStyle="bold"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.112"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.022" />


    <Button
        android:id="@+id/buttonSelectPhoto"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/selectPetPhoto"
        android:background="@color/darkGreen"
        app:layout_constraintBottom_toTopOf="@+id/editTextName"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.498"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/imageDogPlaceholder"
        app:layout_constraintVertical_bias="0.0" />


</androidx.constraintlayout.widget.ConstraintLayout>
```

fragment_rating.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:padding="24dp">


    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Califica al paseador"
        android:textSize="20sp"
        android:textStyle="bold"
        android:layout_marginBottom="20dp"/>


    <RatingBar
        android:id="@+id/ratingBar"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:numStars="5"
        android:stepSize="1"/>


    <Button
        android:id="@+id/buttonSubmitRating"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Enviar"
        android:layout_marginTop="20dp"/>
</LinearLayout>
```

fragment_route.xml

```
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/frameLayoutRoute"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:theme="@style/Theme.AnimalCrossing"
    tools:context=".Route">
```

```
    <TextView
        android:id="@+id/routesTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:text="@string/text1"
        android:textSize="24sp"
        android:textStyle="bold"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.073"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.0" />


    <androidx.recyclerview.widget.RecyclerView
        xmlns:android="http://schemas.android.com/apk/res/android"
        android:id="@+id/routesRecycler"
        android:layout_width="0dp"
        android:layout_height="0dp"
        android:padding="10dp"
        app:layout_constraintTop_toBottomOf="@+id/routesTitle"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintBottom_toBottomOf="parent"/>


</androidx.constraintlayout.widget.ConstraintLayout>
```

fragment_route_map.xml

```
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">


    <!-- MAPA -->
    <fragment
        android:id="@+id/mapFragment"
        android:name="com.google.android.gms.maps.SupportMapFragment"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />


    <!-- Tarjeta de info -->
    <LinearLayout
        android:id="@+id/infoCard"
        android:orientation="vertical"
        android:padding="16dp"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_margin="16dp"
        android:layout_gravity="bottom"
        android:elevation="6dp">


        <TextView
            android:id="@+id/mapTitle"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Detalle de ruta"
            android:textStyle="bold"
            android:textSize="18sp"
            android:paddingBottom="8dp" />


        <TextView
            android:id="@+id/mapPetName"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="" />


        <TextView
            android:id="@+id/mapRouteName"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="" />


        <TextView
            android:id="@+id/mapWalkerName"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="" />


        <Button
            android:id="@+id/buttonFinishWalk"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
```

```
            android:layout_marginTop="12dp"
            android:text="Finalizar paseo" />


    </LinearLayout>


    <TextView
        android:id="@+id/previewText"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Vista previa de la ruta"
        android:textSize="20sp"
        android:textStyle="bold"
        android:padding="12dp"
        android:layout_margin="20dp"
        android:elevation="5dp"
        android:layout_gravity="top|center_horizontal"
        android:visibility="gone" />


</FrameLayout>
```

fragment_walk_in_progress.xml

```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp"
    android:background="@color/background"
    tools:context=".WalkInProgress">


    <TextView
        android:id="@+id/walkTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:text="Paseo en curso"
        android:textColor="@color/black"
        android:textSize="22sp"
        android:textStyle="bold"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.497"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />


    <androidx.cardview.widget.CardView
        android:id="@+id/walkInfoCard"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginTop="24dp"
        app:cardCornerRadius="16dp"
        app:cardElevation="6dp"
        app:layout_constraintTop_toBottomOf="@id/walkTitle"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent">


        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="16dp">


            <TextView
                android:id="@+id/walkPetName"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Mascota: —"
                android:textSize="18sp"
                android:textStyle="bold" />


            <TextView
                android:id="@+id/walkRouteName"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_marginTop="4dp"
                android:text="Ruta: —"
                android:textSize="16sp" />


            <TextView
                android:id="@+id/walkTime"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_marginTop="8dp"
                android:text="Tiempo: 00:00"
```

```
            android:textSize="16sp" />


        <TextView
            android:id="@+id/walkDistance"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="4dp"
            android:text="Distancia: 0.0 km"
            android:textSize="16sp" />


    </LinearLayout>
</androidx.cardview.widget.CardView>



<Button
    android:id="@+id/uploadPhotoButton"
    android:layout_width="168dp"
    android:layout_height="52dp"
    android:layout_marginTop="248dp"
    android:text="Subir foto"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.497"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/walkInfoCard"
    app:layout_constraintVertical_bias="0.0" />


<ImageView
    android:id="@+id/evidenceImage"
    android:layout_width="212dp"
    android:layout_height="193dp"
    android:layout_marginBottom="8dp"
    android:background="@drawable/dog_placeholder"
    android:scaleType="centerCrop"
    app:layout_constraintBottom_toTopOf="@+id/uploadPhotoButton"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.497"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/walkInfoCard"
    app:layout_constraintVertical_bias="0.913" />


<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Evidencia de entrega"
    android:textSize="16sp"
    android:textStyle="bold"
    app:layout_constraintBottom_toTopOf="@+id/evidenceImage"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.493"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/walkInfoCard"
    app:layout_constraintVertical_bias="1.0" />


<Button
    android:id="@+id/finishWalkButton"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_marginBottom="104dp"
    android:background="@color/darkGreen"
    android:text="Finalizar paseo"
    android:textColor="@color/white"
    android:textSize="16sp"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.0"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/uploadPhotoButton" />


</androidx.constraintlayout.widget.ConstraintLayout>
```

fragment_walker_list.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp">


    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Selecciona un paseador"
        android:textSize="20sp"
```

```
        android:textStyle="bold"
        android:layout_marginBottom="12dp"/>


    <TextView
        android:id="@+id/emptyWalkerMessage"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="No hay paseadores disponibles por ahora."
        android:textColor="@color/black"
        android:textSize="18sp"
        android:visibility="gone"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintBottom_toBottomOf="parent"/>


    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/walkersRecycler"
        android:layout_width="match_parent"
        android:layout_height="match_parent"/>


  </LinearLayout>
```

fragment_walker_profile.xml

```
  <ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
      android:layout_width="match_parent"
      android:layout_height="match_parent">


      <LinearLayout
          android:orientation="vertical"
          android:padding="20dp"
          android:layout_width="match_parent"
          android:layout_height="wrap_content">


          <TextView
              android:id="@+id/profileWalkerName"
              android:text="Nombre del paseador"
              android:textSize="22sp"
              android:textStyle="bold"
              android:layout_width="wrap_content"
              android:layout_height="wrap_content"/>


          <TextView
              android:id="@+id/profileWalkerEmail"
              android:text="correo@ejemplo.com"
              android:paddingTop="4dp"
              android:layout_width="wrap_content"
              android:layout_height="wrap_content"/>


          <View
              android:layout_width="match_parent"
              android:layout_height="1dp"
              android:background="#CCCCCC"
              android:layout_marginTop="16dp"
              android:layout_marginBottom="16dp"/>


          <TextView
              android:text="Ruta seleccionada"
              android:textSize="18sp"
              android:textStyle="bold"
              android:layout_width="wrap_content"
              android:layout_height="wrap_content"/>


          <TextView
              android:id="@+id/profileRouteName"
              android:text="Ruta Corta"
              android:layout_width="wrap_content"
              android:layout_height="wrap_content"
              android:paddingTop="6dp"/>


          <TextView
              android:id="@+id/profileRouteTime"
              android:text="10 minutos"
              android:layout_width="wrap_content"
              android:layout_height="wrap_content"
              android:paddingTop="4dp"/>


          <TextView
              android:id="@+id/profileRoutePrice"
              android:text="$40 MXN"
              android:layout_width="wrap_content"
              android:layout_height="wrap_content"
```

```
        android:paddingTop="4dp"/>


    <Button
        android:id="@+id/buttonConfirmWalk"
        android:text="Confirmar paseo"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="30dp"/>


  </LinearLayout>


</ScrollView>
```

item_accepted_requests.xml

```
<androidx.cardview.widget.CardView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:card_view="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="12dp"
    card_view:cardCornerRadius="12dp"
    card_view:cardElevation="6dp">


    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">


        <TextView
            android:id="@+id/acceptedPetName"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Nombre mascota"
            android:textSize="20sp"
            android:textStyle="bold"
            android:textColor="#000000"/>


        <TextView
            android:id="@+id/acceptedOwnerAddress"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Dirección del dueño"
            android:textSize="14sp"
            android:layout_marginTop="6dp"
            android:textColor="#444444" />


        <TextView
            android:id="@+id/acceptedHour"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Hora solicitada"
            android:textSize="14sp"
            android:layout_marginTop="4dp"
            android:textColor="#444444" />


        <Button
            android:id="@+id/startWalkButton"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Iniciar paseo"
            android:layout_marginTop="12dp"
            android:background="@color/darkGreen"
            android:textColor="@color/background"
            android:padding="10dp"/>


    </LinearLayout>


</androidx.cardview.widget.CardView>
```

item_pet_card.xml

```
<androidx.cardview.widget.CardView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_margin="8dp"
    android:elevation="6dp"
    android:padding="8dp"
    tools:theme="@style/Theme.AnimalCrossing">


    <LinearLayout
        android:layout_width="match_parent"
```

```
        android:layout_height="wrap_content"
        android:orientation="horizontal">


        <ImageView
            android:id="@+id/petImage"
            android:layout_width="100dp"
            android:layout_height="100dp"
            android:scaleType="centerCrop"/>


        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:paddingStart="12dp"
            android:paddingEnd="1dp">


            <TextView
                android:id="@+id/petName"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:textStyle="bold"
                android:textSize="18sp"/>


            <TextView
                android:id="@+id/petBreed"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:textSize="14sp"/>


        </LinearLayout>


    </LinearLayout>


</androidx.cardview.widget.CardView>
```

item_route_card.xml

```
<androidx.cardview.widget.CardView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_margin="8dp"
    android:elevation="6dp"
    android:padding="8dp"
    tools:theme="@style/Theme.AnimalCrossing">


    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">


        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="15dp">


            <TextView
                android:id="@+id/routeName"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:textStyle="bold"
                android:textSize="22sp"/>


            <TextView
                android:id="@+id/routePrice"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:textSize="20sp"/>


            <TextView
                android:id="@+id/routeTimeDistance"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:textSize="18sp"
                android:textStyle="italic"/>


            <TextView
                android:id="@+id/moreMapDetails"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
```

```
            android:textSize="16sp"
            android:textStyle="italic"
            android:text="@string/moreMapDetails"/>
        </LinearLayout>


    </LinearLayout>


</androidx.cardview.widget.CardView>
```

item_walk_request.xml

```
<androidx.cardview.widget.CardView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="12dp"
    android:elevation="6dp"
    android:padding="16dp"
    android:backgroundTint="@android:color/white">


    <LinearLayout
        android:orientation="vertical"
        android:layout_width="match_parent"
        android:layout_height="wrap_content">


        <TextView
            android:id="@+id/textOwner"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Dueño:"
            android:textSize="16sp"
            android:textStyle="bold"/>


        <TextView
            android:id="@+id/textPet"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="4dp"
            android:text="Mascota:"
            android:textSize="14sp"/>


        <TextView
            android:id="@+id/textRoute"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="4dp"
            android:text="Ruta:"
            android:textSize="14sp"/>


        <TextView
            android:id="@+id/textStatus"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="6dp"
            android:text="Estado: Pendiente"
            android:textSize="14sp"
            android:textStyle="italic"
            android:textColor="#555555" />


        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal"
            android:gravity="end"
            android:layout_marginTop="10dp">


            <Button
                android:id="@+id/buttonAccept"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Aceptar" />


            <Button
                android:id="@+id/buttonReject"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Rechazar"
                android:layout_marginStart="12dp"/>
        </LinearLayout>


    </LinearLayout>
</androidx.cardview.widget.CardView>
```

item_walker_card.xml

```
<androidx.cardview.widget.CardView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="8dp"
    android:elevation="4dp"
    android:padding="12dp">


    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">


        <TextView
            android:id="@+id/walkerName"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textStyle="bold"
            android:textSize="20sp"/>


        <TextView
            android:id="@+id/walkerEmail"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textStyle="italic"
            android:textSize="14sp"/>


        <TextView
            android:id="@+id/walkerRating"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="⭐ 0.0"
            android:textSize="14sp"
            android:textColor="#FFA000"
            android:layout_marginTop="4dp"/>


    </LinearLayout>


</androidx.cardview.widget.CardView>
```

login.xml

```
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    xmlns:tools="http://schemas.android.com/tools"
    android:background="@color/background"
    tools:theme="@style/Theme.AnimalCrossing">


    <EditText
        android:id="@+id/userPassword"
        android:layout_width="214dp"
        android:layout_height="51dp"
        android:ems="10"
        android:hint="@string/password"
        android:inputType="textPassword"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.497"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.569" />


    <Button
        android:id="@+id/buttonLogin"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:background="@color/darkGreen"
        android:fontFamily="sans-serif-medium"
        android:text="@string/buttonLogin"
        android:textAlignment="center"
        android:textAllCaps="true"
        android:textColor="@color/background"
        android:typeface="normal"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.498"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.672" />


    <EditText
```

```
        android:id="@+id/userEmail"
        android:layout_width="214dp"
        android:layout_height="51dp"
        android:ems="10"
        android:hint="@string/email"
        android:inputType="textEmailAddress"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.497"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.452" />


    <TextView
        android:id="@+id/textView2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/register"
        android:textColor="@color/black"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.496"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/buttonLogin"
        app:layout_constraintVertical_bias="0.425" />


    <Button
        android:id="@+id/buttonRegister"
        android:layout_width="wrap_content"
        android:layout_height="48dp"
        android:background="@color/darkGreen"
        android:fontFamily="sans-serif-medium"
        android:text="@string/buttonRegister"
        android:textAlignment="center"
        android:textAllCaps="true"
        android:textColor="@color/background"
        android:typeface="normal"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/textView2"
        app:layout_constraintVertical_bias="0.2" />


    <androidx.appcompat.widget.AppCompatImageView
        android:id="@+id/imageView2"
        android:layout_width="240dp"
        android:layout_height="140dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.497"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.115"
        app:srcCompat="@drawable/logo" />


    <TextView
        android:id="@+id/textView3"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
        android:text="@string/appName"
        android:textAllCaps="false"
        android:textColor="#242323"
        android:textSize="24sp"
        android:textStyle="bold|italic"
        app:layout_constraintBottom_toTopOf="@+id/userEmail"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/imageView2"
        app:layout_constraintVertical_bias="0.0" />


</androidx.constraintlayout.widget.ConstraintLayout>
```

main_menu.xml

```
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="@color/background"
    android:fitsSystemWindows="true"
    tools:theme="@style/Theme.AnimalCrossing">


    <FrameLayout
        android:id="@+id/fragment_contanier"
```

```
        android:layout_width="match_parent"
        android:layout_height="0dp"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toTopOf="@id/bottomNavigationView"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"/>


    <com.google.android.material.bottomnavigation.BottomNavigationView
        android:id="@+id/bottomNavigationView"
        android:layout_width="match_parent"
        android:layout_height="70dp"
        android:layout_alignParentBottom="true"
        android:layout_marginTop="30dp"
        android:background="@drawable/bottom_background"
        android:elevation="2dp"
        app:itemIconSize="30dp"
        app:itemIconTint="@color/item_selector"
        app:itemRippleColor="@android:color/transparent"
        app:labelVisibilityMode="unlabeled"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.0"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.974"
        app:menu="@menu/bottom_menu" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

register.xml

```
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    xmlns:tools="http://schemas.android.com/tools"
    tools:theme="@style/Theme.AnimalCrossing">


    <Button
        android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/buttonRegister"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.881" />


    <EditText
        android:id="@+id/userPhoneRegister"
        android:layout_width="wrap_content"
        android:layout_height="48dp"
        android:ems="10"
        android:hint="@string/number"
        android:inputType="phone"
        android:maxLength="8"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.497"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.562" />


    <EditText
        android:id="@+id/userNameRegister"
        android:layout_width="wrap_content"
        android:layout_height="48dp"
        android:layout_marginBottom="20dp"
        android:ems="10"
        android:hint="@string/petName"
        android:inputType="text"
        app:layout_constraintBottom_toTopOf="@+id/userAddressRegister"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.497"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.987" />


    <EditText
        android:id="@+id/userEmailRegister"
        android:layout_width="wrap_content"
        android:layout_height="48dp"
        android:ems="10"
        android:hint="@string/email"
        android:inputType="textEmailAddress"
        app:layout_constraintBottom_toTopOf="@+id/userPasswordRegister"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.497"
        app:layout_constraintStart_toStartOf="parent"
```

```
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.826" />


    <EditText
        android:id="@+id/userAddressRegister"
        android:layout_width="wrap_content"
        android:layout_height="48dp"
        android:layout_marginBottom="24dp"
        android:ems="10"
        android:hint="@string/address"
        android:inputType="textPostalAddress"
        app:layout_constraintBottom_toTopOf="@+id/userPhoneRegister"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.497"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.989" />


    <EditText
        android:id="@+id/userPasswordRegister"
        android:layout_width="wrap_content"
        android:layout_height="48dp"
        android:layout_marginBottom="20dp"
        android:ems="10"
        android:hint="@string/password"
        android:inputType="textPassword"
        app:layout_constraintBottom_toTopOf="@+id/userNameRegister"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.497"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="1.0" />


    <RadioGroup
        android:id="@+id/radioGroupRole"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintBottom_toTopOf="@+id/button"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/userPhoneRegister">


        <RadioButton
            android:id="@+id/radioButtonOwner"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="@string/owner" />


        <RadioButton
            android:id="@+id/radioButtonWalker"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="@string/walker" />
    </RadioGroup>


</androidx.constraintlayout.widget.ConstraintLayout>
```

colors.xml

```
<resources>
    <color name="oliveGreen">#BFD962</color>
    <color name="reallyDarkGreenAccent">#356603</color>
    <color name="darkGreenAccent">#488C04</color>
    <color name="darkGreen">#55A605</color>
    <color name="lightGreen">#E3F2AC</color>
    <color name="background">#F2F2F2</color>
    <color name="black">#000000</color>
    <color name="white">#FFFFFF</color>
    <color name="gray">#B3AEAE</color>
</resources>
```

strings.xml

```
<resources>
    <string name="app_name" translatable="false">animalCrossing</string>
    <string name="appName">Animal Crossing</string>
    <!--Navbar-->
    <string name="account">Account</string>
    <string name="home">Home</string>
    <string name="pet">Pet</string>
    <string name="route">Route</string>
    <string name="message">Message</string>
    <!--Login-->
    <string name="email">Correo electrónico</string>
```

```
    <string name="password">Contraseña</string>
    <string name="buttonLogin">Ingresar</string>
    <string name="buttonRegister">Registrarse</string>
    <string name="register">¿Aún no tiene una cuenta? Registrese aquí</string>
    <string name="title_activity_register">Register</string>
    <!--Register-->
    <string name="address">Dirección</string>
    <string name="number">Número telefónico</string>
    <string name="owner">Soy un dueño</string>
    <string name="walker">Soy un paseador</string>
    <!--Main menu-->
    <string name="availablePets">Mascotas registradas</string>
    <string name="title_activity_owner_profile">OwnerProfile</string>
    <string name="userQuestion">¿Qué mascota saldrá a pasear hoy?</string>
    <string name="title_activity_pet">Pet</string>
    <string name="title_activity_routes">Routes</string>
    <string name="title_activity_messages">Messages</string>
    <!--AccountFragment-->
    <string name="saveChangesButton">Guardar Cambios</string>
    <string name="title_account_change">Actualizar Datos</string>
    <string name="title_account_close">Cerrar Sesión</string>
    <!--PetFragment-->
    <string name="title_pet_fragment">Registrar nueva mascota</string>
    <string name="petName">Nombre</string>
    <string name="petBreed">Raza</string>
    <string name="petAge">Edad</string>
    <string name="petDescription">Descripción (Comportamiento, \ncondiciones médicas, etc.)</string>
    <string name="selectPetPhoto">Seleccione una foto</string>
    <string name="registerPet">Registrar mascota</string>
    <!--Route Fragment (and Related)-->
    <string name="text1">Rutas disponibles</string>
    <string name="showRoute">Mostrar recorrido</string>
    <string name="moreMapDetails">\nPresione para ver la ruta en el mapa</string>
    <!-- TODO: Remove or change this placeholder text -->
    <string name="hello_blank_fragment">Hello blank fragment</string>
</resources>
```

themes.xml

```
<resources>
    <style name="Theme.AnimalCrossing" parent="Theme.Material3.Light.NoActionBar" />
    <style name="SlideActivityAnimation" parent="@android:style/Animation.Activity">
        <item name="android:activityOpenEnterAnimation">@anim/slide_in_right</item>
        <item name="android:activityOpenExitAnimation">@anim/slide_out_left</item>
    </style>
</resources>
```

Justificación del Código

Dentro de los aspectos más importantes a destacar del código se tiene el uso de múltiples Fragments sobre usar varias Activities. Un Activity es una pantalla y proceso independiente, mientras que Fragment es un módulo que realiza una acción muy específica y que depende un Activity que lo ejecute. El beneficio de utilizar Fragments sobre Activities es que los Fragments promueven un sistema más modular ya que, como se mencionó, cada Fragment realiza una tarea muy específica.

Por otro lado, los Fragments también facilitan la reutilización de código, por ejemplo, a lo largo del programa se hace uso múltiple de las tarjetas que muestran las rutas disponibles dentro de la aplicación: se muestran al momento de que el usuario quiere ver los detalles de la ruta y se muestran cuando el dueño selecciona una mascota y tiene que elegir una ruta; es decir, se pueden reutilizar los Fragments múltiples veces a lo largo del código, en cambio, si se utilizaran varias Activities esto supondría que se tendría que copiar y pegar el código múltiples veces, además, de no tener cuidado e incluir funciones para destruir o finalizar las Activities, supondría un problema en el rendimiento de la aplicación.

Los Fragments también son mejores al ahorrar recursos; recordando que es una aplicación móvil, el rendimiento es elemental para el funcionamiento correcto de la aplicación, por lo que los Fragments son ideales ya que estos no suelen más que controlar porciones pequeñas de la interfaz de usuario.

Backlog

ID	Nombre	Descripción	Prioridad	Estado	Fecha
1	Creación de App Movil	Se creo la aplicación móvil, además de integrar el Registro de Usuario y Mascotas, Login, Mapas y el diseño de las interfaces básicas.	Muy Alta	Aprobado	06/12/2025
2	Eliminación de tracking de secret.properties	Se eliminó el archivo del repositorio que contenía la API para el uso de GoogleMaps.	Muy Alta	Aprobado	06/12/2025
3	Cambio de flujo de selección de ruta y paseador	Se añadió el flujo básico para que el dueño pudiese elegir una mascota, ruta y paseador.	Alta	Aprobado	07/12/2025
4	Implementación del perfil de Paseador (Walker)	Se implementó el rol de Paseador, además de la opción de Aceptar y Rechazar solicitudes de paseo.	Muy Alta	Aprobado	07/12/2025
5	Arreglo de error en Home Fragment	Se arregló Home.kt, que en ocasiones provocaba que la aplicación se cerrara automáticamente.	Media	Aprobado	07/12/2025
6	Función de Rating de Paseadores	Se añadieron la opción de poder calificar a los paseadores una vez el paseo haya finalizado, además de optimizaciones varias.	Media	Aprobado	14/12/2025
7	Actualización de README	Se actualizó README.md para mostrar las características de la aplicación móvil.	Baja	Aprobado	14/12/2025